

Análise Arquitetural do Sistema — System-As-Is

Projeto de Arquitetura de Software · Análise do projeto psoft-project-2024-g1

Objetivo

Este documento apresenta uma visão geral da estrutura arquitetural do projeto, identificando os principais módulos, componentes, camadas, tecnologias, relações e alguns aspetos relevantes para uma análise de Engenharia de Software e Arquitetura de Software.

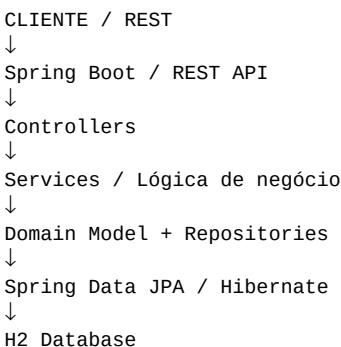
Classificação arquitetural

O sistema pode ser caracterizado principalmente como um **Monólito Modular**, organizado por domínios funcionais. No interior dos módulos é utilizada uma organização em camadas, combinada com princípios associados a **Domain-Driven Design (DDD)**, nomeadamente através da representação explícita de conceitos do domínio e da separação entre domínio, serviços, API, persistência e infraestrutura.

1. Visão geral do sistema

A aplicação é um backend REST desenvolvido em Java com Spring Boot. Não é identificada uma aplicação frontend integrada no projeto; a interação com o sistema é realizada através da API REST, podendo ser utilizada, por exemplo, através do Postman.

Em termos conceptuais, o sistema pode ser representado da seguinte forma:



Ao lado desta estrutura existem preocupações transversais, como autenticação, autorização, armazenamento de ficheiros, tratamento de exceções e funcionalidades partilhadas. Existe também uma integração com um serviço externo.

2. Estrutura global do projeto



A organização é sobretudo orientada a funcionalidades/domínios. Cada domínio mantém no seu interior as classes relacionadas com a respetiva responsabilidade, em vez de concentrar todos os controllers, services e repositories em diretórios globais.

Módulo	Responsabilidade principal
bookmanagement	Gestão de livros, ISBN, títulos, descrições e operações relacionadas.
authmanagement	Gestão de autores e informação associada.
genremanagement	Gestão de géneros e respetivas estatísticas.
readermanagement	Gestão dos leitores, dados pessoais e informação estatística.
lendingmanagement	Gestão de empréstimos, devoluções e multas.
usermanagement	Gestão de utilizadores, papéis e informação de identidade.
auth	Autenticação e emissão/validação de credenciais.
shared	Funcionalidades e conceitos transversais reutilizados por vários módulos.
external	Integrações com serviços externos.
bootstrapping	Inicialização e população de dados.
configuration	Configuração da aplicação e regras configuráveis.
exceptions	Exceções e tratamento centralizado de erros.

3. Organização interna dos módulos

Os principais módulos apresentam uma estrutura interna semelhante, promovendo uma separação entre exposição da API, lógica de aplicação, modelo de domínio, abstração de persistência e implementações de infraestrutura.

```
bookmanagement/
  ■■■ api/
  ■■■ model/
  ■■■ services/
  ■■■ repositories/
  ■■■ infrastructure/
```

API — contém controllers, views e objetos de entrada/saída utilizados na comunicação REST.

Services — concentra operações e lógica de aplicação/negócio, incluindo pedidos e consultas.

Model — representa conceitos do domínio, incluindo entidades e value objects.

Repositories — define abstrações para acesso aos dados.

Infrastructure — contém implementações tecnológicas concretas, como adaptadores baseados em Spring Data.

4. Modelo de domínio

O domínio da aplicação é centrado numa biblioteca. Entre os principais conceitos encontram-se Book, Author, Genre, Reader, Lending e Fine. Existem também value objects específicos, como Isbn, Title, Description, Name, BirthDate, PhoneNumber e EmailAddress.

A existência destes objetos específicos permite representar conceitos do domínio de forma mais explícita do que através de tipos genéricos como String ou Integer em todas as situações.

```
Reader
■
■■■ Lending ■■■ Book
■
```

■■■ Fine

Book ■■■ Genre

Book ■■■ Author

5. Principais módulos funcionais

5.1 Book Management

Responsável pela gestão de livros. Inclui Book, Isbn, Title e Description, bem como controllers, serviços, pedidos, consultas, views e repositories.

5.2 Author Management

Responsável pelos autores e respetiva informação. Inclui operações de gestão e funcionalidades relacionadas com livros, coautores e estatísticas.

5.3 Genre Management

Responsável pelos géneros literários e por informação estatística relacionada com empréstimos e quantidade de livros.

5.4 Reader Management

Responsável pelos leitores, incluindo Reader, ReaderNumber, EmailAddress, PhoneNumber e BirthDate, bem como informação e operações estatísticas relacionadas com empréstimos.

5.5 Lending Management

Responsável pelos empréstimos e devoluções. Inclui Lending, LendingNumber, Fine, pedidos para criar empréstimos e marcar devoluções, e pesquisas de empréstimos.

5.6 User Management e Authentication

O User Management trata da identidade dos utilizadores e dos papéis, incluindo Reader e Librarian. O módulo de autenticação integra-se com Spring Security e JWT para autenticação e autorização.

6. Fluxo arquitetural típico

```
HTTP Request
↓
Controller / API
↓
Service
↓
Domain Model / regras de negócio
↓
Repository
↓
Spring Data / Hibernate
↓
H2 Database
```

O controller funciona como ponto de entrada da API. O serviço coordena a operação e as regras necessárias. O repository abstrai o acesso aos dados, sendo a implementação concreta fornecida pela infraestrutura tecnológica.

7. Persistência e infraestrutura

A persistência utiliza Spring Data JPA e Hibernate sobre uma base de dados H2. Os repositories do domínio são separados das implementações tecnológicas existentes na infraestrutura.

Esta separação permite reduzir o acoplamento direto dos serviços relativamente à tecnologia de persistência e facilita a identificação dos pontos onde a infraestrutura pode ser substituída.

8. Segurança

O projeto utiliza Spring Security, JWT e papéis de utilizador, distinguindo pelo menos os perfis `READER` e `LIBRARIAN`. O fluxo conceptual é:

Credenciais → autenticação → JWT → pedido REST com Bearer Token → Spring Security → autorização por papel

9. Funcionalidades transversais

O módulo `shared` concentra conceitos e serviços reutilizáveis, incluindo funcionalidades associadas a fotografias, armazenamento de ficheiros, concorrência, nomes e outras utilidades. Existe ainda um módulo de exceções para tratamento de erros.

Esta abordagem evita duplicação, embora seja importante controlar o crescimento do módulo `shared` para evitar que se transforme num ponto de acoplamento excessivo entre os restantes módulos.

10. Integração externa

O projeto contém um serviço denominado `ApiNinjasService`, responsável pela comunicação com uma API externa. A integração encontra-se isolada no módulo `external`, evitando que os restantes componentes tenham de conhecer diretamente os detalhes da API externa.

11. Configuração e regras de negócio

Existem propriedades configuráveis relacionadas com regras da biblioteca, como a duração dos empréstimos, valor da multa por dia, idade mínima do leitor e limites de sugestões por género. A externalização destas regras reduz a necessidade de alterar código para modificar determinados parâmetros operacionais.

12. Testes e qualidade

O projeto possui testes unitários e testes de integração. Existem testes orientados ao domínio, como `BookTest`, `AuthorTest`, `GenreTest`, `LendingTest` e `ReaderTest`, bem como testes de `repositories`, `services` e `controllers`.

A existência de testes em diferentes níveis permite validar tanto unidades isoladas como a integração entre componentes e infraestrutura.

13. Caracterização arquitetural

Monólito Modular

Todos os módulos fazem parte da mesma aplicação e do mesmo processo de execução, mas encontram-se organizados por domínios funcionais.

Arquitetura em camadas

Dentro dos módulos existe separação entre API, serviços, domínio, repositories e infraestrutura.

Princípios de DDD

O projeto representa conceitos do domínio através de entidades e value objects específicos.

REST

A interação externa é realizada através de endpoints HTTP disponibilizados pela aplicação.

14. Vista arquitetural consolidada



15. Pontos arquiteturais relevantes para análise

Modularidade: a organização por domínio favorece a coesão funcional e torna mais fácil localizar o código associado a uma determinada capacidade do sistema.

Isolamento da infraestrutura: os repositories possuem abstrações separadas das implementações baseadas em Spring Data.

Base de dados partilhada: sendo um monólito, os diferentes módulos utilizam a mesma infraestrutura de persistência. Isto simplifica o deployment, mas reduz o isolamento relativamente a uma arquitetura de microserviços.

Shared: fornece reutilização transversal, mas deve ser controlado para evitar dependências excessivas entre módulos.

Integrações externas: a existência de um adaptador específico para o serviço externo ajuda a limitar o acoplamento com esse fornecedor.

Segurança: a autenticação baseada em JWT e a autorização por papéis constituem uma preocupação transversal relevante para a arquitetura.

Gestão de segredos: a configuração do projeto deve ser revista para garantir que chaves, credenciais e outros segredos não são mantidos diretamente no código ou repositório. Em ambiente real, estes valores deverão ser externalizados e geridos por mecanismos apropriados de configuração e gestão de segredos.

16. Síntese para o documento de arquitetura

O sistema analisado apresenta uma arquitetura de monólito modular orientada ao domínio. A solução organiza as principais capacidades da biblioteca em módulos funcionais, nomeadamente gestão de livros, autores, géneros, leitores, empréstimos e utilizadores. Cada módulo combina uma API de acesso, serviços de aplicação, modelo de domínio, abstrações de persistência e componentes de infraestrutura.

A utilização de Spring Boot, Spring Data JPA, Hibernate, Spring Security e JWT fornece a infraestrutura tecnológica necessária para disponibilizar a API REST, persistir dados e controlar o acesso. A existência de componentes partilhados e de um adaptador para serviços externos complementa a arquitetura.

Do ponto de vista de Arquitetura de Software, os principais elementos a estudar posteriormente são as dependências entre módulos, os limites de cada contexto funcional, os mecanismos de comunicação, o grau de acoplamento, a coesão interna, as responsabilidades dos componentes e os potenciais pontos de evolução da arquitetura.

Documento preparado com base na análise do projeto fornecido para a unidade curricular de Arquitetura de Software.